

Dynamic Programming

→ optimization problems

↳ (minima & maxima)

→ overlapping subproblems

Fibonacci series — 1 1 2 3 5 8 ...

Fib(n) :

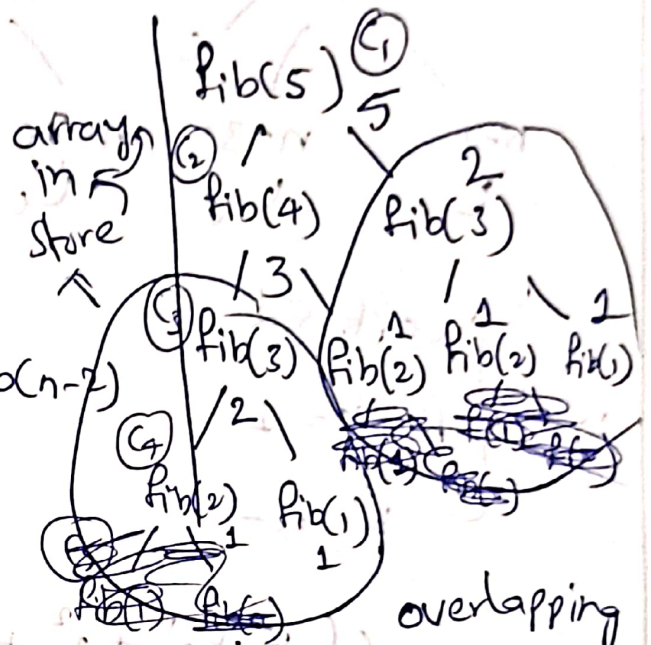
if $n=1$ or $n=2$

return 1

otherwise

result = $\text{Fib}(n-1) + \text{Fib}(n-2)$

return result



→ so, instead of doing again & again store that in array and check, this is called

1) [memoization]

↳ or Top down approach

→ In Mergesort

↳ No overlapping subproblem

↳ so, Divide & Conquer can be applied

2) Tabulation (bottom up approach)

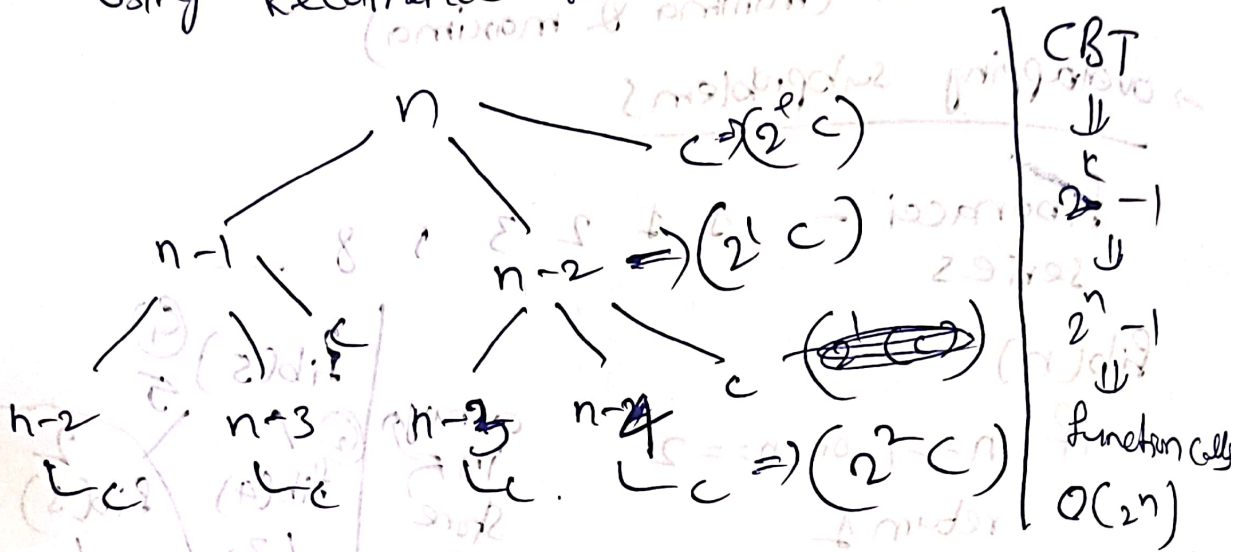
1	2	3	4	5	6
1	1	2	3	5	8

↳ Not using any recursion

Time Complexity for Recursion in Fibonacci

$$T(n) = T(n-1) + T(n-2) + C$$

Using Recurrence Relation



Left side
 $n - k = 1$
 $k = n - 1$

Right side
 $n - 2k = 1$
 $k = \frac{n-1}{2}$

$$C(2^0 + 2^1 + 2^2 + \dots + 2^k) \quad k = n$$

$$C(2^0 + 2^1 + 2^2 + \dots + 2^n) \quad \text{Max}$$

GP series
 $r = 2 > 1$

$$S = \frac{a(r^n - 1)}{r - 1} = \frac{2^n - 1}{2 - 1} \Rightarrow \frac{2^n - 1}{1}$$

$O(2^n)$ $\rightarrow$ exponential Time complexity

Fib(5)



Fib(4)



Fib(3)



Fib(2)



Fib(1)

} $\Rightarrow O(n)$

↳ from

exponential Time complexity to Linear Time Complexity
(Recursion)

(Dynamic Programming)
↓
Memoization Tabulation



recursion

```
def fib(n):
```

```
    if n == 1 or n == 2:
```

```
        return 1
```

```
    return fib(n-1) + fib(n-2)
```

fib(5), fib(100)

↓
takes long time

⇒ TC is $O(2^k) \Rightarrow O(2^n)$

memoization (top down approach)

```
def fib-topdown(n, memo):
```

```
    if memo[n] is not None:
```

```
        return memo[n]
```

```
    if n == 1 or n == 2:
```

```
        result = 1
```

```
    else:
```

```
        result = fib-topdown(n-1, memo) + fib-topdown(n-2, memo)
```

```
    memo[n] = result
```

```
    return result
```

```
def fib-memo(n):
```

```
    memo = [None] * (n+1)
```

```
    return fib-topdown(n, memo)
```

fib-memo(5)

fib-memo(100)

fib-memo(1000)

fib-memo(1500)

↓
takes time
because of recursion